南開大學
Nankai University

计算机学院

并行程序设计实验报告

CPU 架构相关编程

姓名： 崔迪

学号： 2412439

专业： 计算机科学与技术

课程： 并行程序设计

指导教师： 崔立骁

完成日期： 2026 年 4 月 1 日

目录

一、 基础要求 **3**
（一） 算法设计 3
1. $n*n$ 矩阵与向量内积 3
2. n 个数求和 3
（二） 编程实现 4
1. $n*n$ 矩阵与向量内积 4
2. n 个数求和 4
（三） 性能测试 4
1. $n*n$ 矩阵与向量内积 4
2. n 个数求和 6
（四） profiling 7
1. $n*n$ 矩阵与向量内积 7
2. n 个数求和 7
（五） 结果分析 8
1. $n*n$ 矩阵与向量内积 8
2. n 个数求和 9

二、 进阶要求 **10**
（一） 循环展开优化 10
1. 算法设计 10
2. 性能测试 11
3. profiling 11
4. 与原算法对比 11
（二） 其他可选进阶优化（待补充） 12

三、 实验总结和思考 **12**
（一） 异同对比 12
（二） 总结和思考 12

摘要

本实验围绕 CPU 架构相关编程中的两个基础问题展开：一是 $n \times n$ 矩阵与向量内积，二是 n 个数求和。针对矩阵与向量内积，分别实现了按列访问的平凡算法和按行访问的 cache 优化算法；针对求和问题，分别实现了链式累加、两路链式累加以及两两相加的递归算法三种实现。实验采用固定数据构造方式，以高精度计时器对不同问题规模下的程序平均运行时间进行测量，并将结果导出为 CSV 文件后进一步绘图分析。

实验结果表明：对于矩阵与向量内积问题，若按实验平台信息中给出的 cache 容量选择“未充满 L1、未充满 L2、未充满 L3、基本充满 L3、溢出 L3”五个阶段的规模，按行访问的 cache 优化算法始终优于按列访问的平凡算法；当矩阵规模增至 $n = 2048$（32 MiB，已明显超过 24 MiB 的 L3）时，相较平凡算法获得约 23.00 倍加速。对于 n 个数求和问题，两路链式累加始终优于平凡链式累加，说明减少累加依赖链长度、提升指令级并行能力是有效的。递归算法虽然在理论上具有更短的依赖深度，但在当前单线程实现中由于存在额外复制和多轮中间结果写回，其实际性能未超过两路链式累加。进阶部分进一步给出了循环展开优化的设计思路，并为后续的性能测试与 profiling 留出了补充位置。

本次实验的代码和结果均在`https://github.com/changluei/Parallel-Programming`

实验平台信息

项目	说明
操作系统	Microsoft Windows 11 家庭中文版（10.0.26200）
处理器	13th Gen Intel(R) Core(TM) i7-13650HX，14 核 20 线程，最高时钟频率 2.60 GHz
CPU 各级 cache	L1:1.2 MB L2:11.5 MB L3:24.0 MB
内存	13.70 GiB
编译器	`g++`（MinGW），主要编译选项为 `-O2 -std=c++17`
绘图环境	Conda 环境 `parallel`，Python 配合 `matplotlib`、`pandas`、`numpy`
profiling 工具	Intel VTune Profiler 2025.10
数据组织	原始实验数据以 CSV 文件形式保存在 `results` 目录中，绘图结果保存在 `fig` 目录中

一、 基础要求

（一） 算法设计

1. $n * n$ 矩阵与向量内积

设输入向量为 $a \in \mathbb{R}^n$，矩阵为 $B \in \mathbb{R}^{n \times n}$，输出向量为 $c \in \mathbb{R}^n$。本实验计算矩阵每一列与向量 a 的内积，即

$$c_j = \sum_{i=0}^{n-1} B_{i,j} a_i, \qquad j = 0, 1, \ldots, n-1.$$

本实验实现了两种算法。

1. **平凡算法（逐列访问）**：外层循环按列遍历，内层循环按行累加，即访问顺序为 `B[row * n + col]`。由于矩阵在内存中按行连续存放，这种按列跨步访问会频繁跨越 cache line，属于 cache 不友好的访问方式。

2. **cache 优化算法（逐行访问）**：先将结果向量清零，然后按行遍历矩阵，将当前行的每个元素与对应的 $a[row]$ 相乘并直接累加到结果向量中。该实现与矩阵的行主序存储方式一致，内层循环访问连续内存，更利于 cache 预取与空间局部性利用。

两种算法的数学结果等价，但由于访存模式不同，在现代处理器上会表现出明显不同的运行时间与 cache 行为。

2. n 个数求和

对于数组 $x[0], x[1], \ldots, x[n-1]$，目标是计算

$$S = \sum_{i=0}^{n-1} x[i].$$

本实验实现了以下三种求和方式。

1. **平凡算法（链式累加）**：维护单个累加器 `sum`，按顺序执行 `sum += x[i]`。该方法实现最简单，但加法之间形成一条很长的数据依赖链。

2. **优化算法（两路链式累加）**：维护两个独立累加器 `sum1` 和 `sum2`，分别累加奇偶位置，循环结束后再合并两个部分和。该方法可以打断单一依赖链，更有利于超标量处理器挖掘指令级并行。

3. **递归思想（两两相加）**：先将相邻两个元素相加得到 $n/2$ 个中间结果，再对中间结果继续两两相加，重复进行 $\log_2 n$ 轮，直到剩下一个结果。本次实验采用二重循环方式实现。

其中，两路链式累加和递归思想都属于对处理器执行特性的利用：前者重点减少串行依赖、增强指令级并行，后者则从规约树结构上缩短依赖深度。

（二） 编程实现

1. $n*n$ 矩阵与向量内积

为了便于实验比较，程序使用固定规则自动生成输入向量和矩阵，为便于测量不同规模的数据，本次采用 vector 实现。由于篇幅限制原因，本次不再展示平凡算法的实现。

```
//优化算法实现
fill(sum.begin(), sum.end(), 0.0);
for (int row = 0; row < n; ++row) {//逐行访问矩阵元素
    const double factor = a[row];
    const size_t offset = static_cast<size_t>(row) * n;
    for (int col = 0; col < n; ++col) {
        sum[col] += b[offset + col] * factor;
    }
}
```

Listing 1: 矩阵与向量内积的优化算法核心实现

2. n 个数求和

本次同样采用固定规则生成数据。其核心求和代码如下。

```
//两路链式累加实现
double sum1 = 0.0;double sum2 = 0.0;int i = 0;
const int n = static_cast<int>(data.size());
for (; i + 1 < n; i += 2) {
    sum1 += data[i];
    sum2 += data[i + 1];
}
for (; i < n; ++i) {
    sum1 += data[i];
}
return sum1 + sum2;
//两两相加的递归算法实现，采用二重循环实现
copy(input.begin(), input.end(), work.begin());
for (int m = static_cast<int>(input.size()); m > 1; m /= 2) {
    for (int i = 0; i < m / 2; ++i) {
        work[i] = work[i * 2] + work[i * 2 + 1];
    }
}
return work[0];
```

Listing 2: 两种求和优化核心实现

（三） 性能测试

1. $n*n$ 矩阵与向量内积

本实验采用 chrono::steady_clock 作为计时工具，对各算法的核心计算部分进行高精度计时。steady_clock 具有单调递增、不受系统时间调整影响的特点，适合用于性能测试。测试时将算法主体封装为独立函数，在主程序中对同一规模下的算法重复执行多次，并以总耗时除以重复次数得到平均运行时间 avg_ms，从而减小单次测量误差带来的影响。矩阵实验不再采用简单倍增的规模序列，而是按照实验平台表中给出的 L1=1.2 MiB、

L2=11.5 MiB、L3=24.0 MiB 选点，以观察“未充满 L1、未充满 L2、未充满 L3、基本充满 L3、溢出 L3”五个阶段的性能变化。综合“尽量采用 2^n 的幂规模”和“需要覆盖不同 cache 阶段”两点考虑，最终选取 $n = 256, 1024, 1536, 1792, 2048$，对应矩阵大小分别约为 0.50 MiB、8.00 MiB、18.00 MiB、24.50 MiB 和 32.00 MiB。由于不同规模下的运行时间差异较大，重复次数分别设置为 4000、500、250、150 和 100。实验结果如图 1 所示，详细数据见表 1。

规模 n	矩阵大小（MiB）	平凡算法（ms）	cache 优化算法（ms）	加速比
256	0.50	0.032270	0.020948	1.54
1024	8.00	3.844896	0.310954	12.36
1536	18.00	11.484792	1.481136	7.75
1792	24.50	15.754887	1.888080	8.34
2048	32.00	53.424360	2.322430	23.00

表 1: 矩阵与向量内积性能测试结果

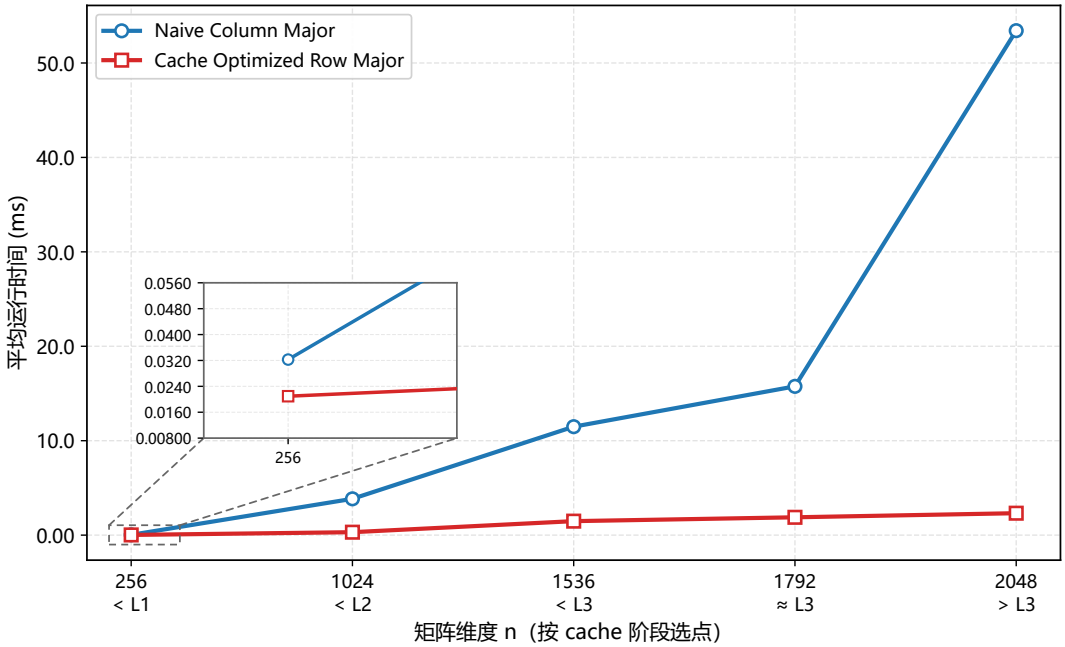

图 1: 矩阵与向量内积运行时间折线图

从图中可以看到，按 cache 阶段选点后，矩阵规模从未充满 L1 逐步增大到溢出 L3 时，平凡算法与 cache 优化算法之间的差距整体上明显拉大，尤其在 24 MiB 附近继续增大到 32 MiB 时出现了更显著的性能跳升。相关具体分析将会在后文展开。

2. n 个数求和

求和实验同样选择规模 $n = 2^{10}, 2^{11}, 2^{12}, 2^{13}, 2^{14}$，即 1024 到 16384。由于单次求和耗时极短，为了提高计时稳定性，每个数据点均重复执行 1000 次并取平均值。实验结果如图 2 和表 2 所示。

规模 n	平凡链式（ms）	两路链式（ms）	递归算法（ms）	平凡/两路	平凡/递归
1024	0.000411	0.000214	0.000345	1.92	1.19
2048	0.000890	0.000452	0.000587	1.97	1.52
4096	0.001754	0.000855	0.001323	2.05	1.33
8192	0.003638	0.001730	0.003193	2.10	1.14
16384	0.007332	0.003606	0.006953	2.03	1.05

表 2: n 个数求和性能测试结果

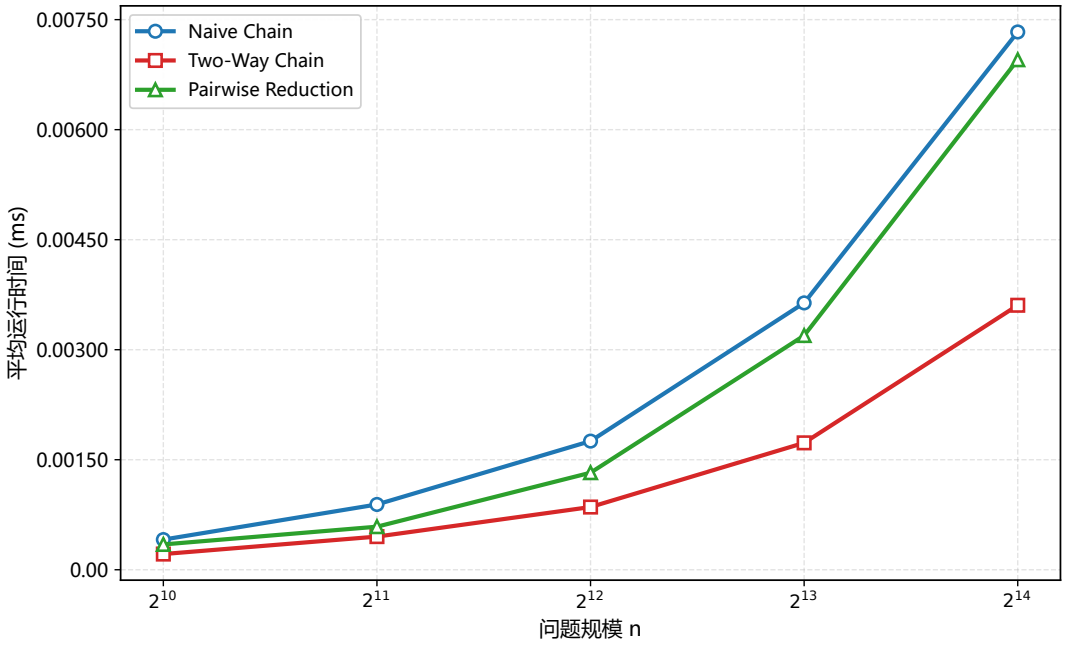

图 2: n 个数求和运行时间折线图

在该实验中，三种算法都能够得到一致的求和结果，但由于其数据依赖关系和中间数据处理方式不同，运行时间存在显著差异。其中，两路链式累加整体最优，说明通过增加独立累加链可以有效提升指令级并行度。至于为何递归算法未能超过两路链式累加，主要原因会在后文进行剖析。

（四） profiling

1. $n * n$ 矩阵与向量内积

对于该实验，使用 VTune 对其三级 cache 的 hit/miss 进行实验和记录。理论上，平凡算法的热点应集中在按列访问的内层循环中，并且会出现更明显的缓存失配；优化算法则由于按行顺序访问矩阵，热点仍在核心循环，但 cache 行为应明显优于平凡算法。本次实验均选用 n=16384 的规模举例进行 profiling，以突出两种算法在大规模数据下的访存行为差异。.

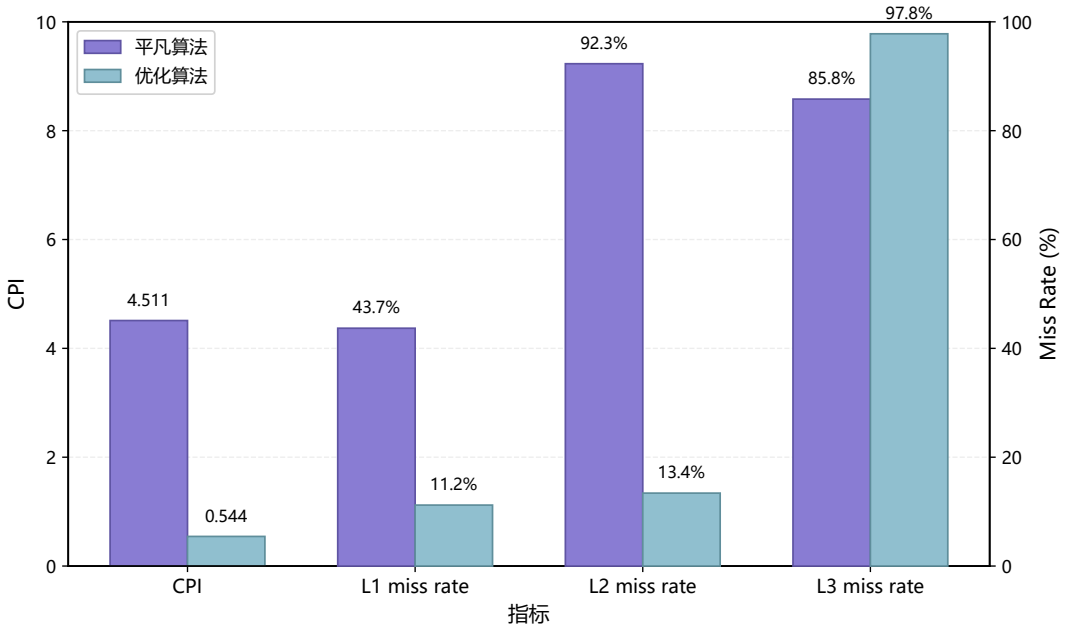

图 3: 矩阵与向量内积 profiling 结果图例

版本	CPI	L1 hit	L1 miss	L1 miss rate	L2 hit	L2 miss	L2 miss rate	L3 hit	L3 miss	L3 miss rate
平凡算法	4.511	9.240×10^8	7.164×10^8	43.7%	5.480×10^7	6.597×10^8	92.3%	9.262×10^7	5.611×10^8	85.8%
优化算法	0.544	1.084×10^9	1.364×10^8	11.2%	1.176×10^8	1.820×10^7	13.4%	4.001×10^5	1.761×10^7	97.8%

表 3: 矩阵与向量内积 profiling 结果记录表

分析将在后文中结果分析部分展开。

2. n 个数求和

对于求和实验，本文采用 VTune 的与 `Microarchitecture Exploration` 分析类型，重点关注热点函数、执行指令数 (Instructions Retired)、时钟周期数 (Clockticks)、CPI 以及 CPU time 等指标。平凡链式累加通常会因为单条累加依赖链较长而限制指令级并行；两路链式累加有望在保持较低实现成本的同时显著提升吞吐；递归算法则可能在热点中表现出额外的数组复制和多轮规约开销。

版本	热点函数	Instructions Retired	Clockticks	CPI Rate	CPU Time (ms)
平凡链式	`naiveChainSum`	4.20×10^7	3.64×10^7	0.867	7.991
两路链式	`twoWayChainSum`	3.92×10^7	1.68×10^7	0.429	3.995
递归算法	`pairwiseReductionSum`	1.12×10^8	1.96×10^7	0.175	6.992

表 4: n 个数求和 profiling 结果记录表

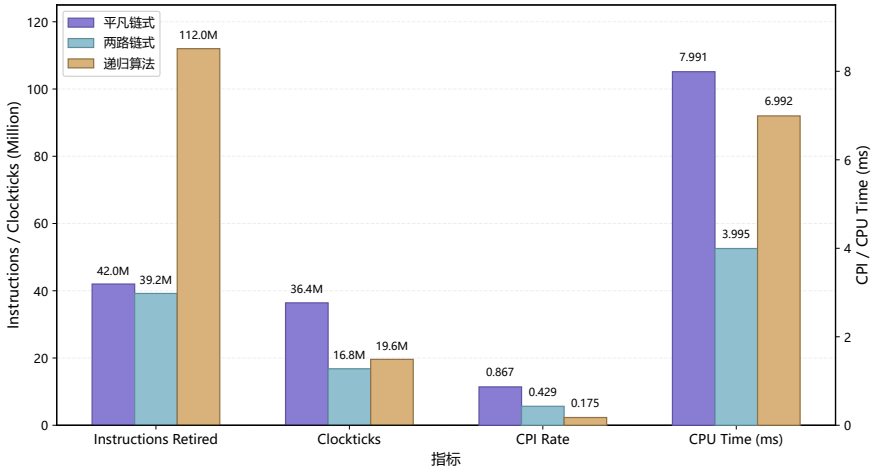

图 4: n 个数求和 profiling 结果图例

（五） 结果分析

1. $n * n$ 矩阵与向量内积

从表 1 和图 1 可以看出，按照“未充满 L1、未充满 L2、未充满 L3、基本充满 L3、溢出 L3”五个阶段选点后，矩阵与向量内积的性能变化更容易和 cache 容量联系起来理解。具体而言，当 $n = 256$ 时，矩阵仅为 0.50 MiB，尚未充满报告中给出的 1.2 MiB L1 容量，此时平凡算法和优化算法的平均运行时间分别为 0.032270 ms 与 0.020948 ms，加速比仅为 1.54 倍，说明在工作集较小时，访存模式差异虽然存在，但尚未被 cache 层次结构明显放大。

当规模增大到 $n = 1024$ 时，矩阵大小达到 8.00 MiB，已经远超 L1 但仍低于报告中给出的 11.5 MiB L2 总容量。此时平凡算法耗时已增至 3.844896 ms，而优化算法仅为 0.310954 ms，加速比达到 12.36 倍。之所以在“未充满 L2”的阶段差距就已经非常明显，一方面是因为按列访问本身会严重破坏空间局部性，导致 cache line 利用率很低；另一方面，报告中的 L1/L2 容量是处理器总体口径，而单线程程序实际能够稳定利用的有效 cache 空间通常小于总量，因此性能拐点会早于“总容量上限”完全体现出来。

继续增大到 $n = 1536$ 时，矩阵大小约为 18.00 MiB，已经超过 L2 但仍未充满 24 MiB 的 L3；到 $n = 1792$ 时，矩阵大小约为 24.50 MiB，已经基本触及 L3 容量边界。对

应地，平凡算法运行时间由 11.484792 ms 进一步上升到 15.754887 ms，而优化算法仅由 1.481136 ms 增加到 1.888080 ms，说明随着工作集逼近 L3，上层 cache 难以容纳全部跨步访问带来的代价逐渐被放大，但按行访问版本仍能依靠连续访存保持较稳定的增长趋势。

最明显的变化出现在 $n = 2048$ 时。此时矩阵大小为 32.00 MiB，已经明显超过 24 MiB 的 L3 容量，平凡算法的平均运行时间跃升到 53.424360 ms，而优化算法仅为 2.322430 ms，加速比达到 23.00 倍。也就是说，从“基本充满 L3”到“溢出 L3”这一步，按列访问版本出现了最强烈的性能恶化，这与实验设计中希望观察到的 cache 分层效应是一致的。

平凡算法按列访问矩阵，连续两次访问的地址间隔为 n 个 `double`，在行主序存储下属于典型的跨步访问。这样做不仅破坏了空间局部性，也使得硬件预取器难以稳定发挥作用，很多 cache line 在只使用其中极少数元素后就被替换掉。相比之下，优化算法按行扫描矩阵，内层循环连续访问 `b[offset + col]`，既符合内存布局，也更利于 cache line 被充分利用；同时单个 `a[row]` 会在该行处理中被反复复用，因而数据供给更加平滑。这说明对于该问题，决定性能的关键并不是乘加次数，而是是否顺应了 CPU 的存储层次结构。

profiling 结果进一步验证了这一点。优化版本的 CPI 从 4.511 降低到 0.544，说明处理器等待数据的周期显著减少，流水线能够以更高效率执行有效指令。进一步观察 miss rate，可见优化算法的 L1 miss rate 由 43.7% 降到 11.2%，L2 miss rate 由 92.3% 降到 13.4%。这与运行时间曲线的变化趋势是一致的，即按行访问后，大量数据在更高层 cache 中就已经被命中，不再频繁掉入更低层存储。

需要说明的是，优化算法的 L3 miss rate 仍然较高，但这并不否定优化效果。原因在于：对于 $n = 16384$ 这类超大规模数据，矩阵整体远大于片上 cache，程序本身仍带有明显的流式访存特征，因此部分访问最终仍会落到内存层面。优化的价值并不是让所有访问都命中 L3，而是尽可能把可利用的局部性留在 L1/L2 阶段，从而显著降低平均访存代价。综合性能测试与 profiling 结果可以得出结论：矩阵与向量内积实验的主要瓶颈来自 cache 不友好的访存模式，而按行访问的优化正是围绕这一瓶颈展开的。

2. n 个数求和

从表 2 和图 2 可以看出，三种算法的运行时间都随 n 近似线性增长，但两路链式累加始终表现最好，在所有测试规模下都稳定优于平凡链式累加，加速比约为 1.92 到 2.10 倍。这一结果与课程中关于超标量处理器的分析一致：平凡链式累加只有一个累加器，`sum += x[i]` 之间形成了很长的数据依赖链，后一条加法必须等待前一条加法完成；而两路链式累加将其拆成 `sum1` 和 `sum2` 两条相互独立的链，使处理器能够在同一时间推进更多指令，因此吞吐率更高。又由于本实验中数组规模仅为 8 KiB 到 128 KiB，远小于 L2 cache 容量，且访问模式始终是顺序扫描，所以该问题的主要矛盾并不在 cache miss，而在于依赖链是否限制了指令级并行。

pairwise reduction 的表现则呈现出一个很有代表性的“理论更优、实测不一定更快”

的情况。仅从依赖图看，两两相加形成的是规约树，其依赖深度约为 $\log_2 n$，明显短于平凡链式的 n 级依赖，因此直觉上似乎应该更快。但本实验中的实现方式是**单线程、标量、迭代式**规约：每次求和都要先把输入数组复制到工作数组 `work` 中，然后进行多轮中间结果写回。这样一来，算法虽然缩短了“结果产生路径”的深度，却额外引入了大量内存读写和循环控制开销。

这一点可以从 profiling 数据中看得更清楚。pairwise reduction 的 CPI 只有 0.175，甚至低于两路链式累加的 0.429,看起来流水线利用得很好;但它退休的指令数达到 112,000,000，约为两路链式累加 39,200,000 的 2.86 倍。也就是说，pairwise reduction 不是“每条指令执行得慢”，而是“总共执行了太多指令”，因此其总 CPU time 仍为 6.992 ms，明显高于两路链式累加的 3.995 ms。这也说明仅看 CPI 这类单位指标是不够的，必须结合总指令数和总运行时间一起分析。

从实现细节看，两路链式累加只需对原数组做一次线性扫描，两个部分和大多可以长期保存在寄存器中，最后再做一次合并；而 pairwise reduction 不仅有一次完整复制，还要对 `work` 数组执行 $\log_2 n$ 轮规约，每一轮都会重复读取和写回中间结果。对于当前这种单线程 CPU 实现，额外的数据搬运成本足以抵消规约树在依赖深度上的理论优势，因此曲线没有两路链式累加“好看”，并且在规模增大后逐渐接近平凡链式累加。需要补充的是，本实验测试的 n 均为 2 的幂，这已经避开了 pairwise reduction 在非 2 的幂规模下处理尾元素的额外复杂度；即便在这一相对理想的条件下，它仍未超过两路链式累加，更说明当前瓶颈主要来自实现常数而非理论深度。

因此，可以得出如下结论：对于单线程求和问题，最有效的优化方式并不是简单追求更短的理论规约深度，而是选择能够以较低额外开销打破依赖链的实现。两路链式累加恰好在“依赖链缩短”和“实现代价很低”之间取得了较好平衡；而 pairwise reduction 更适合在多线程并行规约、SIMD 向量化或树形归并场景下发挥优势，在当前实验设定下则不如两路链式累加稳定。

二、 进阶要求

（一） 循环展开优化

1. 算法设计

在基础实验之上，可以继续对核心循环进行循环展开（loop unrolling）优化，以进一步减少循环控制开销并提升处理器吞吐率。针对本实验中的两个问题，可以采用如下思路。

1. 对矩阵与向量内积的按行访问版本，将内层循环按 4 或 8 个元素展开，使一次循环迭代中完成多次乘加操作，从而减少分支与索引更新次数。

2. 对 n 个数求和的两路链式累加版本继续展开，例如扩展为 4 路甚至 8 路累加器，进一步增强独立加法链数量，为超标量发射和编译器自动向量化创造条件。

可参考如下伪代码。

```
// 以 4 路链式累加为例
sum0 = sum1 = sum2 = sum3 = 0;
for (i = 0; i + 3 < n; i += 4) {
    sum0 += a[i];
    sum1 += a[i + 1];
    sum2 += a[i + 2];
    sum3 += a[i + 3];
}
sum = sum0 + sum1 + sum2 + sum3;
```

Listing 3: 循环展开优化示意代码

2. 性能测试

本节建议在基础实验的相同规模下，增加循环展开版本的独立测试结果，并使用与前文一致的 CSV 导出和折线图展示方式，以便和原始实现直接对比。由于当前仓库中尚未加入循环展开版本的可执行程序，故本节先给出表格模板，待后续补充实测数据。

问题类型	规模 n	原算法（ms）	展开后（ms）	加速比	备注
矩阵与向量内积	【待填】	【待填】	【待填】	【待填】	【待填】
n 个数求和	【待填】	【待填】	【待填】	【待填】	【待填】

表 5: 循环展开优化性能测试记录表（待补充）

3. profiling

建议对循环展开前后的程序再次进行 VTune 分析，重点比较指令退休数量、前端开销以及后端停顿占比的变化，以验证循环展开是否真的减小了循环控制成本，并观察编译器是否在展开基础上进一步做了自动向量化。

【此处插入循环展开优化前后 profiling 对比截图】

4. 与原算法对比

与原算法相比，循环展开优化理论上应具有如下优势：

1. 减少循环分支判断和计数器更新次数，降低控制开销；
2. 增加同一基本块内的独立运算数量，使处理器更容易挖掘指令级并行；

3. 在某些场景下为编译器自动向量化提供更好的条件。

但循环展开也可能带来代码体积增加、寄存器压力上升等问题，因此最终效果仍需以实际测试和 profiling 数据为准。**【待补充：结合后续实测数据填写最终结论】**

（二） 其他可选进阶优化（待补充）

除循环展开外，本实验还可以进一步从以下几个方向扩展：

1. 使用 SSE/AVX 等 SIMD 指令实现向量化；
2. 对比不同编译优化级别（如 `-O0`、`-O2`、`-O3`）对性能的影响；
3. 分析不同求和顺序对浮点结果误差的影响；
4. 结合 Godbolt 查看关键循环的汇编代码生成情况。

本节可根据课程进度或个人实验完成情况继续补充，当前先保留为扩展接口。

三、 实验总结和思考

（一） 异同对比

本次两个基础实验都围绕“算法实现与 CPU 架构特性的关系”展开，但二者所体现的重点不同。矩阵与向量内积实验主要反映存储层次结构的重要性：核心差异不在算术操作次数，而在于访问矩阵时是否遵循了内存连续布局；n 个数求和实验则主要体现指令级并行的重要性：核心差异不在访存模式，而在于是否打破了单条累加依赖链，让处理器能够同时推进更多独立指令。

二者的共同点在于：都不能只从“算法步骤数”出发评价性能，还必须结合具体处理器的 cache、流水线、超标量发射等硬件特性进行分析。也正因为如此，性能测试和 profiling 不能相互替代，前者告诉我们“快了多少”，后者帮助我们理解“为什么会快”。

（二） 总结和思考

通过本次实验，我更加直观地认识到“相同数学结果，不同实现方式可能对应完全不同的实际性能”这一事实。对于矩阵与向量内积问题，只是改变访问顺序就能带来数量级上的性能差异；对于求和问题，看似简单的累加操作也会因为依赖链组织方式不同而表现出明显不同的执行效率。这说明在并行与高性能程序设计中，理解底层硬件结构并围绕其特性组织代码，是实现性能优化的关键。

后续若继续完善本实验，建议重点补充以下内容：一是完成循环展开版本的独立实现与测试；二是结合 VTune 的具体数据，将 cache miss、热点函数、停顿来源与前文的性能曲线进行一一对应；三是进一步尝试 SIMD 向量化与编译器优化级别对比，从而形成更完整的 CPU 架构相关优化实验链条。**【此处也可补充个人在实验过程中的问题、改进思路与心得体会】**